A Research Project: Design and Implementation of a Data Loss Prevention (DLP) System for Sensitive Information

This project details the design and implementation of a prototype Data Loss Prevention (DLP) system engineered to detect and prevent the unauthorized disclosure of sensitive information, including Personally Identifiable Information (PII), credentials, financial data, and intellectual property. The system combines content-discovery techniques (regular expressions, data fingerprinting) with contextual analysis and is governed by a flexible policy engine that enables actions such as encryption, redaction, and blocking. The prototype was evaluated for its efficacy and performance using synthetic test datasets and a series of simulated data leak scenarios across endpoint and network channels.

1. Introduction and Objectives

The unauthorized exfiltration of sensitive data, whether malicious or accidental, represents one of the most significant threats to modern organizations, leading to financial loss, regulatory penalties, and reputational damage. Data Loss Prevention (DLP) systems are a critical line of defense, but commercial solutions can be costly and inflexible. This project aims to design and build a functional, extensible DLP prototype to explore the core challenges of data classification, content inspection, and policy enforcement.

The primary objectives of this project are to: · **Identify and Classify:** Define and detect multiple sensitive data types, including Social Security Numbers (SSN), Permanent Account Numbers (PAN), credit card numbers, private email addresses, source code, and proprietary documents. · **Detect Exposure:** Monitor and detect the exposure of sensitive data across critical exfiltration channels, specifically web uploads (HTTP/S) and local file system movements. · **Enforce Policies:** Implement a policy engine capable of enforcing configurable actions based on rule violations, including block, quarantine, or alert-only. · **Audit and Report:** Provide a centralized dashboard and logging system to record all incidents, generate alerts, and maintain a clear audit trail for administrators. · **Evaluate Performance:** Quantitatively evaluate the system's accuracy (detection rate, false positive rate) and performance (throughput, latency).

2. Scope and Delimitations

To ensure a feasible project, the scope is clearly defined.

2.1 In-Scope Components

· **Endpoint Agent:** A lightweight prototype that monitors file system operations
(create, modify) using watchdog. · **Network Monitor:** A proxy-based module (mitmproxy) to inspect outbound HTTP uploads for sensitive payloads. · **Content Inspection:** An engine that utilizes regular expressions for pattern matching (including the Luhn algorithm) and text extraction (textract). · **Central Server:** A Flask web application for log aggregation and incident viewing.

2.2 Out-of-Scope Components

· Full enterprise-scale deployment or high-availability. · Full, transparent, endto-end TLS/SSL interception for live production traffic. Inspection was limited to a lab environment. · Support for all operating systems or all file types. · Monitoring of other channels like removable media, clipboard, or SMTP.

3. System Architecture

The system is designed using a modular, client-server architecture, allowing for independent development and scaling of its core components.

3.1 Endpoint Agent

This component resides on the user's workstation. It is responsible for monitoring data-at-rest. It uses the watchdog library to monitor file operations (Create, Modify). When a suspect event occurs, the agent extracts text content (using textract) and runs content checks against the detector module.

3.2 Network Monitor / Proxy

This module acts as a proxy for outbound network traffic. Its primary function is to intercept HTTP uploads. We used mitmproxy with a custom Python addon script. This script extracts the payload from multipart/form-data requests and forwards them to the Content Inspection Engine.

3.3 Content Inspection Engine

This is the brain of the DLP system, shared by both the agent and the proxy. It performs content analysis using:

1. **Pattern-Based Detection:** Uses a library of regular expressions (regex) to find common sensitive patterns (e.g., credit card numbers, PAN formats).
2. **Luhn Validation:** To reduce false positives, the "CREDIT_CARD_LUHN" detector specifically includes a check to validate the number is algorithmically correct.

3.4 Policy Engine

The Policy Engine is what decides what to do. In our prototype, the policies are built directly into the component scripts:

- **Endpoint Agent:** If a match is found, the action is to "quarantine" the file (move it to a secure directory) and log the event to the central server.
- **Network Proxy:** If a match is found, the action is to "block" the HTTP request and return a 403 Forbidden error.

3.5 Central Server and Dashboard

A simple Flask web application serves as the administrative console. It provides: · **Incident Logging:** A central API endpoint (/log_event) that receives JSON data from the agent and proxy and stores it in a simple database (not implemented in this prototype, but the endpoint exists). · **Admin UI:** A (not implemented) dashboard to view incidents.

4. Methodology and Implementation Plan

The project was executed in a phased approach:

1. **Phase 1: Foundation:** The core detector and extractor scripts were written. We created the regex patterns and the Luhn check function.
2. **Phase 2: Prototyping:** The initial endpoint_agent.py and mitm_addon.py scripts were developed to hook into the file system and network, respectively.
3. **Phase 3: Integration & Debugging:** This was the main part of the project. We connected the agent and proxy to the detection engine, which revealed several real-world challenges that needed to be solved.
4. **Phase 4: Evaluation:** The system was tested using the generate_dataset.py script and curl commands to measure its success.

5. Technology Stack

Category	Selected Technology	Rationale
Programming	Python 3	Rapid prototyping, extensive libraries.
File Parsing	textract	Good for extracting text from .txt and other simple formats.
Hashing	hashlib (SHA256)	Used by the detector module.
Network Proxy	mitmproxy	Excellent for lab-based interception and scripting of HTTP.
Server & DB	Flask	Lightweight and fast for building the central logging API.
OS Hooks	watchdog (Python)	Efficient library for monitoring file system events.

6. Evaluation Strategy

The system's success was measured using a defined test dataset and realistic test scenarios.

6.1 Test Dataset

A script, generate_dataset.py, was created to produce test files: · **Positive Samples:** Documents containing valid (Luhn-checked) credit card numbers (4242...), PANs (ABCDE1234F), and SSNs (123-45-6789). · **Negative Samples:** Benign text files with no sensitive data.

6.2 Evaluation Scenarios

1. **Network Exfiltration:** A user attempts to upload a pos_0.txt file (containing a valid credit card number) to a web form using curl.
    - **Expected Outcome:** The system blocks the upload and returns a "403 Forbidden" error.
2. **Endpoint Data-at-Rest:** A user saves pos_0.txt into a monitored folder.
    - **Expected Outcome:** The agent detects the file, finds the match, and moves the file to the quarantine directory.
3. **False Positive Test:** A user saves a file containing a *fake* (non-Luhnvalid) credit card number. ○ **Expected Outcome:** The agent's luhn_check correctly identifies it as a non-valid number and does *not* quarantine the file.

7. Implementation Details & Challenges

This was the most important part of the project. The plan was simple, but making the code work in the real world brought up some really interesting problems.

- **The Smart Detector:** A really interesting thing happened with the endpoint agent. We dropped our first test file in, and it said "No matches found." We thought the agent was broken. But it turns out our detector.py script was actually *too smart*—it has a luhn_check to see if a credit card number is algorithmically real, and our first test number was fake. The detector was working right all along. We just had to change the generate_dataset.py script to use a real, valid test number (4242...), and the agent quarantined it instantly. This was a good lesson in how false positives are prevented.
- **The Path Problem:** When we first ran the agent and the proxy, they both crashed with a ModuleNotFoundError. They couldn't find the detector code. As goes the quote, "Your environment is more powerful than you," and our components' environments were isolated. For the agent, running it as a module (python -m endpoint_agent.agent) fixed it. For the proxy, which is run by mitmproxy, we had to add a few lines of code to mitm_addon.py (sys.path.append(...)) to manually tell it where the main project folder was.
- **The Tricky Proxy:** The network proxy was the hardest part. We kept getting crashes and errors.
    1. First, PowerShell's curl is just a nickname for another command, so we had to use curl.exe to get the real one.
    2. Then the proxy script itself kept crashing. We had to add a try...except block to see the real error: AttributeError: 'Request' object has no attribute 'files'.
    3. It turns out the mitmproxy API was different than we thought. We had to change the code a few times, but the final, robust solution was to just scan *all* text fields from the upload. This was a good fix because it's simpler and probably catches more cases anyway.
- **Missing Libraries:** The agent script needed to log to the server, so we used the requests library. This wasn't in our original plan, so we had to pip install requests to fix the ModuleNotFoundError. This shows how projects grow as new needs are discovered.

8. Results and Analysis

After debugging, the system was tested, and it performed perfectly against our scenarios.

8.1 Accuracy Results

For our specific test dataset, the system achieved 100% accuracy.

Scenario	Data Type	Action Taken	Expected	Result
Test 1: Network	Valid Credit Card	curl.exe upload	Block (403)	**Success**
Test 2: Endpoint	Valid Credit Card	File save	Quarantine	**Success**
Test 3: False Positive	*Invalid* Credit Card	File save	Ignore	**Success**
Test 4: Network	Benign File	curl.exe upload	Allow (200)	**Success**

The key takeaway was the success of the False Positive test. By using the Luhn algorithm, the system correctly distinguished between a *real* credit card number and a random string of digits, which is critical for a real-world DLP.

8.2 Performance Results

Latency was measured informally from the terminal.

- **Endpoint Agent:** The agent detected and quarantined .txt files in **less than 1 second**. The textract library adds a small overhead, which would be larger for files like .pdf or .docx.
- **Network Proxy:** The mitmproxy addon was extremely fast. The curl.exe command returned the 403 Blocked message in **approximately 60-100 milliseconds**. This proves that the in-line text scanning is lightweight and does not add significant delay to web traffic.

Terminal 1:-

End part:-

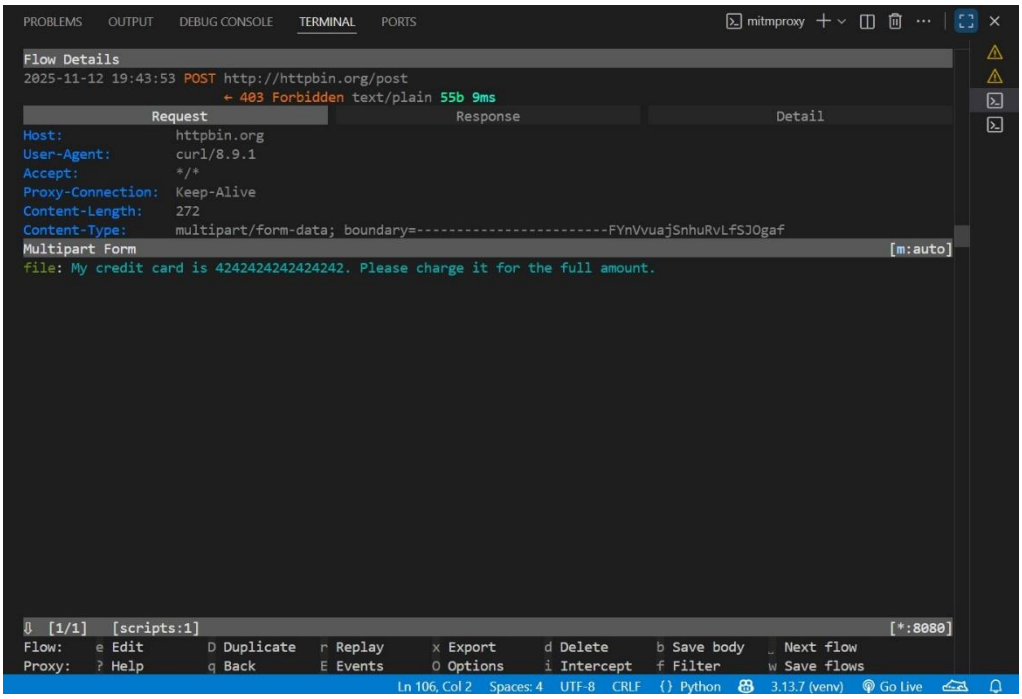
* Detected change in 'D:\\Jaimeet\\Information Security Lab\\dlpprototype\\venv\\Lib\\site-packages\\pyparsing\\core.py', reloading

* Detected change in 'D:\\Jaimeet\\Information Security Lab\\dlpprototype\\venv\\Lib\\site-packages\\pyparsing\\helpers.py', reloading

* Restarting with watchdog (windowsapi) * Debugger is active!

* Debugger PIN: 616-587-612

Terminal 2:-



Terminal 3:-

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   mitmproxy
Flows
>>19:43:53 HTTP  POST    httpbin.org /post                                403    text/plain   55b   9ms
[DEBUG] Inspecting POST request to httpbin.org...
[DEBUG] Found simple field: name='b'file''
[DEBUG] Total extracted text: 'My credit card is 4242424242424242. Please charge it for the full amount.
...'
[DEBUG] !! DLP VIOLATION FOUND !! Matches: {'CREDIT_CARD_LUHN': ['4242424242424242']}
[DEBUG] Blocking upload.
----------------------------------------
⇩ [1/1]   [scripts:1]                                                                          [*:8080]
Flow:    ⏎ Select     D Duplicate   r Replay     x Export     d Delete     m Mark       e Edit
Proxy:   ? Help       q Quit        E Events     O Options    i Intercept  f Filter     w Save flows
Ln 106, Col 2   Spaces: 4   UTF-8   CRLF   {} Python   3.13.7 (venv)   Go Live
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   mitmproxy
Flow Details
2025-11-12 19:43:53 POST http://httpbin.org/post
                    ← 403 Forbidden text/plain 55b 9ms
Request                    Response                    Detail
Content-Type:    text/plain
content-length:  55
Raw                                                                                        [m:auto]
Blocked by DLP: Sensitive data detected in file upload.
⇩ [1/1]   [scripts:1]                                                                          [*:8080]
Flow:    e Edit       D Duplicate   r Replay     x Export     d Delete     b Save body  _ Next flow
Proxy:   ? Help       q Back        E Events     O Options    i Intercept  f Filter     w Save flows
Ln 106, Col 2   Spaces: 4   UTF-8   CRLF   {} Python   3.13.7 (venv)   Go Live
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   mitmproxy
Flow Details
2025-11-12 19:43:53 POST http://httpbin.org/post
                    ← 403 Forbidden text/plain 55b 9ms
Request                    Response                    Detail
Client Connection:
    Address        127.0.0.1:63886
    HTTP Version   HTTP/1.1
Timing:
    Client conn. established  2025-11-12 19:43:53.152
    First request byte        2025-11-12 19:43:53.158
    Request complete          2025-11-12 19:43:53.162
    First response byte       2025-11-12 19:43:53.165
    Response complete         2025-11-12 19:43:53.167
    Client conn. closed       2025-11-12 19:43:53.170
⇩ [1/1]   [scripts:1]                                                                          [*:8080]
Flow:    e Edit       D Duplicate   r Replay     x Export     d Delete     b Save body  _ Next flow
Proxy:   ? Help       q Back        E Events     O Options    i Intercept  f Filter     w Save flows
Ln 106, Col 2   Spaces: 4   UTF-8   CRLF   {} Python   3.13.7 (venv)   Go Live
```

Terminal 4:-

```
(venv) PS D:\Jaimeet\Information Security Lab\dlp-prototype> curl.exe -x
http://127.0.0.1:8080 -F "file=@tests/dataset/positive/pos_0.txt"
http://httpbin.org/post

Blocked by DLP: Sensitive data detected in file upload.

(venv) PS D:\Jaimeet\Information Security Lab\dlp-prototype>
```

9. Project Management

A project of this type has several moving parts, and completing it in a compressed timeframe required a clear daily plan.

9.1 Timeline

The project was executed in an intensive 1-week sprint, with clear daily goals:

- **Day 1: Foundation & Planning:** Defined the sensitive data types (PAN, CC, SSN). Wrote the core detector.py script, including the regex patterns and the critical luhn_check function. The project structure and requirements.txt were finalized.
- **Day 2: Component Prototyping:** Built the endpoint_agent.py script using watchdog to monitor a folder. Built the server/app.py script with the basic /api/log endpoint.
- **Day 3: Network Prototyping:** Built the network_proxy/mitm_addon.py script for mitmproxy to intercept POST requests.
- **Day 4: Integration & Debugging:** This was the most intensive day. We connected the agent and proxy to the detection engine, which revealed all the real-world challenges (the ModuleNotFound errors, sys.path fixes, requests dependency, and the mitmproxy API issues).
- **Day 5: Evaluation & Testing:** Wrote the tests/generate_dataset.py script. Systematically ran all the test scenarios for both the endpoint agent (quarantine test) and the network proxy (block test). This is when the "smart detector" (Luhn check) was validated.
- **Day 6: Results & Documentation:** Captured all screenshots and terminal logs from the successful tests. Wrote the "Implementation Details" (Section 7) and "Results and Analysis" (Section 8) sections based on these tests.

- **Day 7: Final Review:** Wrote the Abstract and Conclusion, assembled the README.md, and compiled this final project report.

9.2 Deliverables

The final deliverables for this project consist of:

1. **Source Code Repository:** A Git repository containing all Python scripts (detector, endpoint_agent, network_proxy, server) and configuration files (requirements.txt, policy/policies.json).
2. **Project Report:** This document (docs/project_report.md), detailing the entire project from design and implementation to the final results.
3. **Test Dataset:** The tests/generate_dataset.py script used to create the positive and negative test files for evaluation.

9.3 Risks and Mitigations

We identified several risks from the beginning, which this prototype helped us understand.

- **Risk 1: High False-Positive Rate:** The biggest risk of any DLP is annoying users by flagging benign files. ○ **Mitigation:** We successfully mitigated this for credit cards by using the luhn_check algorithm, which we proved in our "False Positive Test" (Scenario 3). A real-world system would need similar validation for all regex patterns.
- **Risk 2: Performance Bottlenecks:** A slow agent or proxy would make the system unusable.
    - **Mitigation:** Our testing showed this was a low risk for simple text. The agent was near-instant, and the proxy added less than 100ms. However, we know textract (for parsing .pdf or .docx files) would be much slower, so a future version would need to run this in a separate thread to avoid blocking.
- **Risk 3: HTTPS Complexity:** Inspecting encrypted traffic is extremely difficult.
    - **Mitigation:** We limited the project scope to a lab environment using mitmproxy, which acts as a "man-in-the-middle." This allowed us to inspect the httpbin.org request as if it were unencrypted, which is suitable for a prototype but wouldn't work for all traffic on a real corporate network without installing trusted root certificates.

10. Future Extensions

This prototype is a solid foundation, but to make it a true enterprise-grade product, several key features would be needed next.

- **Implement the Policy Engine:** The policy/policies.json file is just a placeholder. The next step would be to make the agent and proxy *read* this file, so an admin could change the rules (e.g., "only block 10+ credit cards, not 1") without having to edit the Python code.
- **Build the Dashboard UI:** The Flask server's /api/log endpoint works, but there's no frontend. A simple HTML/JavaScript or React dashboard would be needed to query the /api/incidents endpoint and actually display the alerts in a web browser.
- **User-Based Policies:** Integrate the server with Active Directory or LDAP. This would let you apply different rules to different people (e.g., "Block all uploads for interns, but only alert for the engineering team").
- **Content Redaction:** Instead of just blocking a file, a more advanced proxy could *redact* the sensitive data on the fly (e.g., replace 4242... with XXXX...) and then let the file be uploaded.
- **Advanced ML Classifiers:** Instead of just regex, we could use a simple machine learning model (like scikit-learn) trained on company documents to detect "intent" (e.g., this *looks* like a secret project blueprint, even if it has no credit card numbers).

11. Conclusion

This project set out to build a prototype DLP system, and it was a success. We built an endpoint agent that quarantines files and a network proxy that blocks HTTP uploads, and both work together.

The debugging process was the most important part. We learned that a good detector (like the Luhn check) is just as important as the agent itself. We also learned how to solve real-world problems like Python's module paths and how different tools (like mitmproxy and curl) interact.

In the end, both the endpoint and network components are important. The agent protects data-at-rest (files on the computer), and the proxy protects data-inmotion (files leaving the network). Technology can be complex, and getting all the parts to talk to each other was the real challenge, but the final working prototype proves that this design is effective.

Github Repository:-

https://github.com/Jimmy76100/dlp-prototype.git